Procesos de Desarrollo de Software

CLASE 2 – CLASES DE SOFTWARE

Agenda de la clase

- Clases
- Clases estáticas
- Relaciones entre las clases
- Elementos de una relación
- Clases abstractas e interfaces

Clases

- Este tipo de elementos normalmente se utilizan para representar conceptos o entidades del "**negocio**".
- Una clase define un **grupo de objetos** que comparten características, condiciones y significado.

+ Pública

Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.

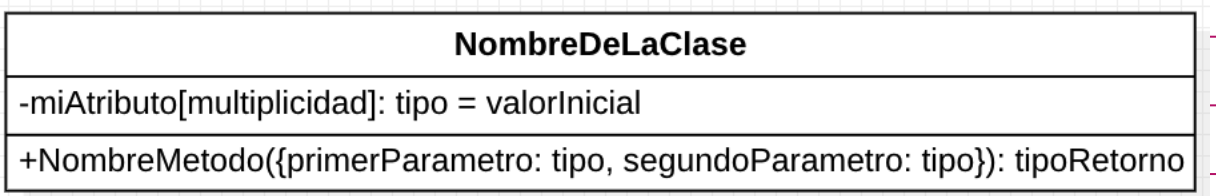
- Privada

Representa que se puede acceder al atributo o función únicamente desde la misma clase.

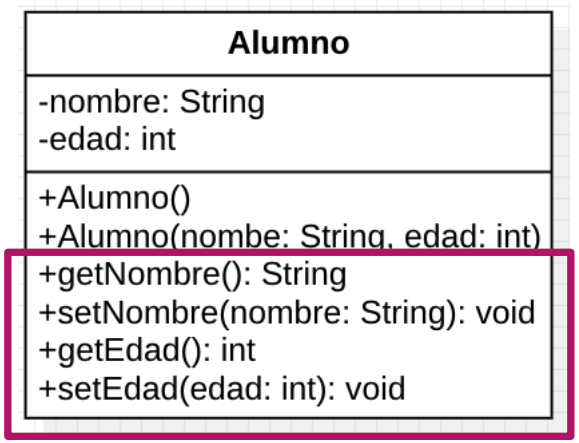
Protegida

Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).

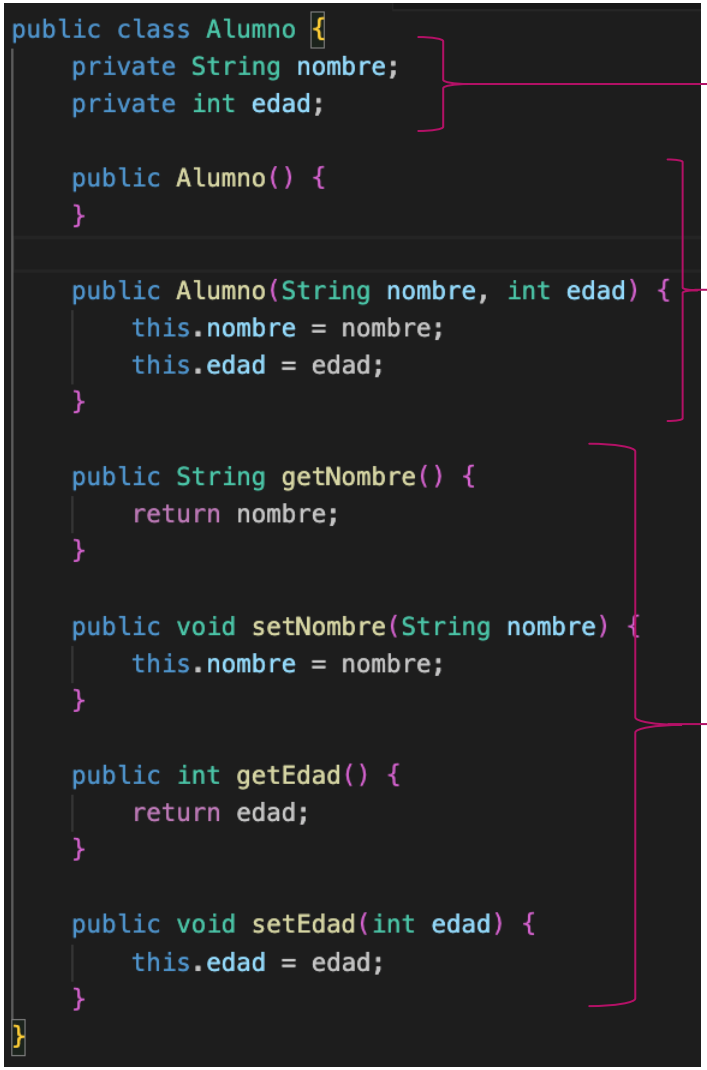
Representación de una clase



- **Ejemplo**



- **siempre vamos a modelar los atributos privados para controlar la forma en que se acceden a los mismos.*



Atributos (privados por default)

Constructores

Getters y Setters

¿Qué es un constructor?

- Un **constructor** es un método especial que se ejecuta automáticamente cuando se crea un objeto.
- Sirve para **inicializar el estado del objeto** (sus atributos).

Características clave:

- Tiene **el mismo nombre que la clase**
- **No tiene tipo de retorno** (ni ***void***)
- Se ejecuta con **new**
- Puede estar **sobrecargado** (varios constructores).

Constructores sin parámetro

```
public Alumno() {
}
```

- No inicializa nada
- Es el constructor por defecto
- Los atributos quedan con valores por defecto:
  - nombre = null
  - edad = 0

- Modo de uso

```
Alumno a1 = new Alumno();
a1.setNombre("Juan");
a1.setEdad(20);
```

Constructores con parámetros

```java
public Alumno(String nombre, int edad) {
    this.nombre = nombre;
    this.edad = edad;
}
```

- Inicializa el objeto directamente con los valores pasados por parámetros.
- **this** referencia al objeto actual.

- Modo de uso

```java
Alumno a2 = new Alumno("Ana", 22);
```

Objetos

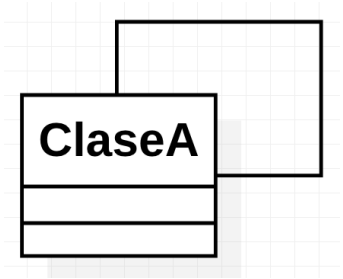
- Un objeto es una instancia de una clase.
- Representa una entidad concreta.
- Los objetos tienen estado propio.
- Todo objeto cuenta con tres características que lo diferencia del resto:
  - **Identidad**
  - **Comportamiento**
  - **Estado Interno**

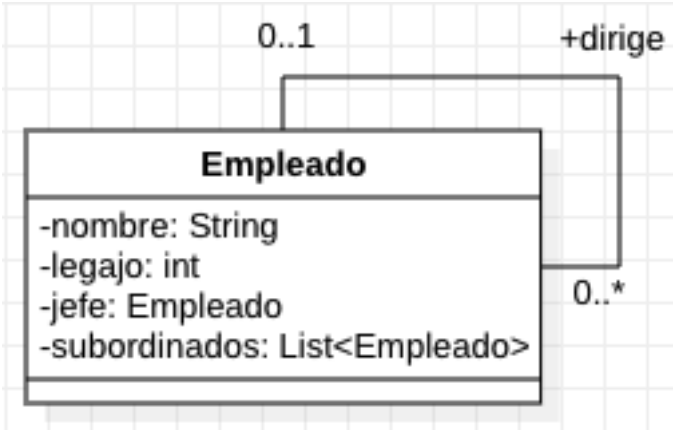
Relaciones entre Clases

- El propósito de la asociación puede expresarse en un nombre, verbo o frase que describa como los objetos de un tipo (clase) se relacionan con objetos de otro tipo (clase). Por ejemplo:
  - Una persona **tiene** un coche
  - Una persona **maneja** un coche
- **Multiplicidad**: cuantos objetos van a participar en la relación

Relaciones entre Clases

- Una relación **identifica una dependencia**. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacía sí misma (menos común, pero existen), este último tipo de dependencia se denomina **dependencia reflexiva**.



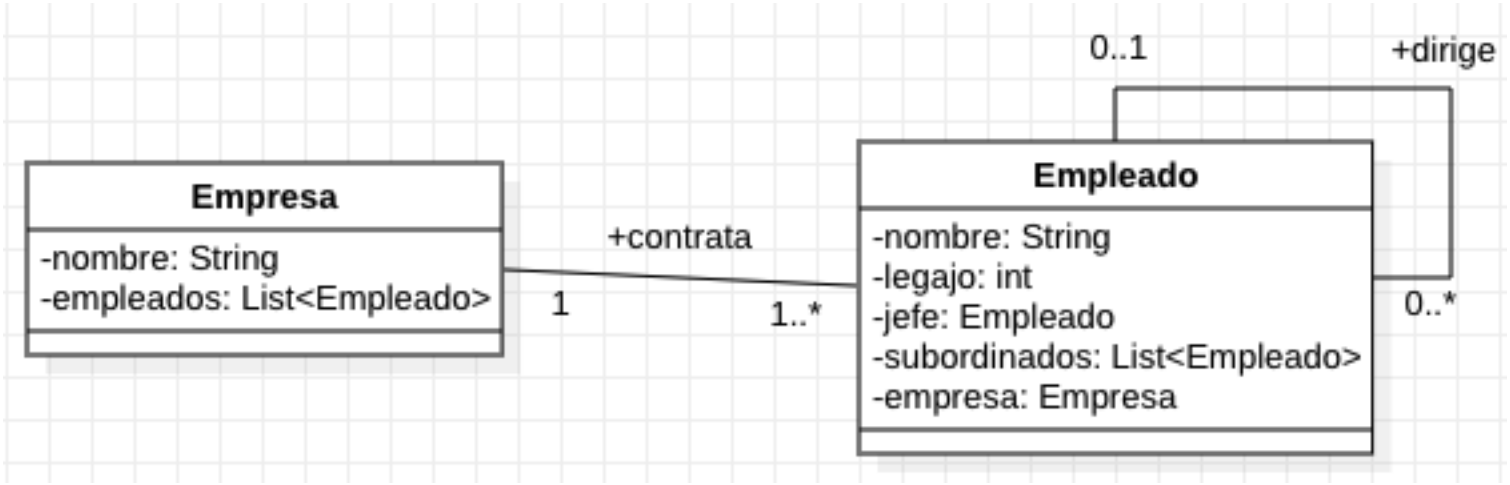


Elementos de una relación

- **Multiplicidad**
  - Número de elementos de una clase que participan en una relación. Se puede indicar un número, un rango… Se utiliza 'n' o '*' para indicar varios.
  - La multiplicidad que se coloca en un extremo indica cuántas instancias de **ESA** clase puede tener la otra.
- **Nombre de la asociación**
  - Ayuda a entender la relación que tienen dos clases. Suelen utilizarse verbos como por ejemplo: "Una empresa **contrata** a **n** empleados".

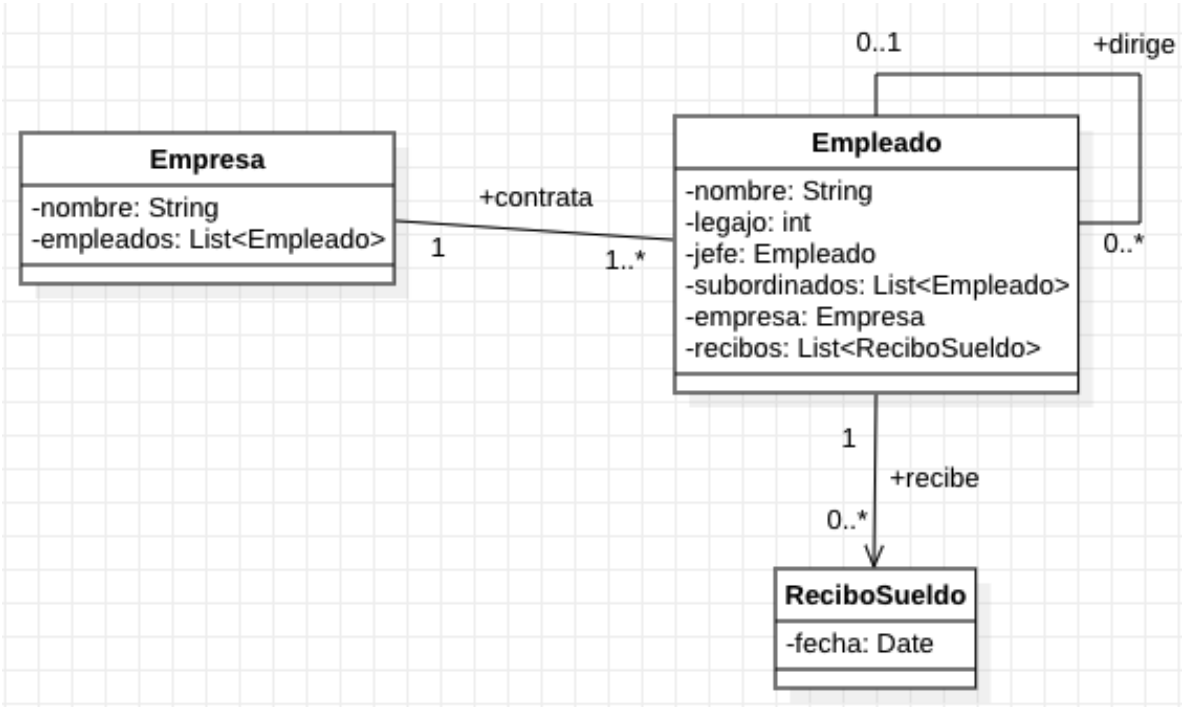
Relación bidireccional

- Una relación **bidireccional** es aquella donde:
  - Ambas clases se conocen entre sí
  - Se puede navegar en ambos sentidos



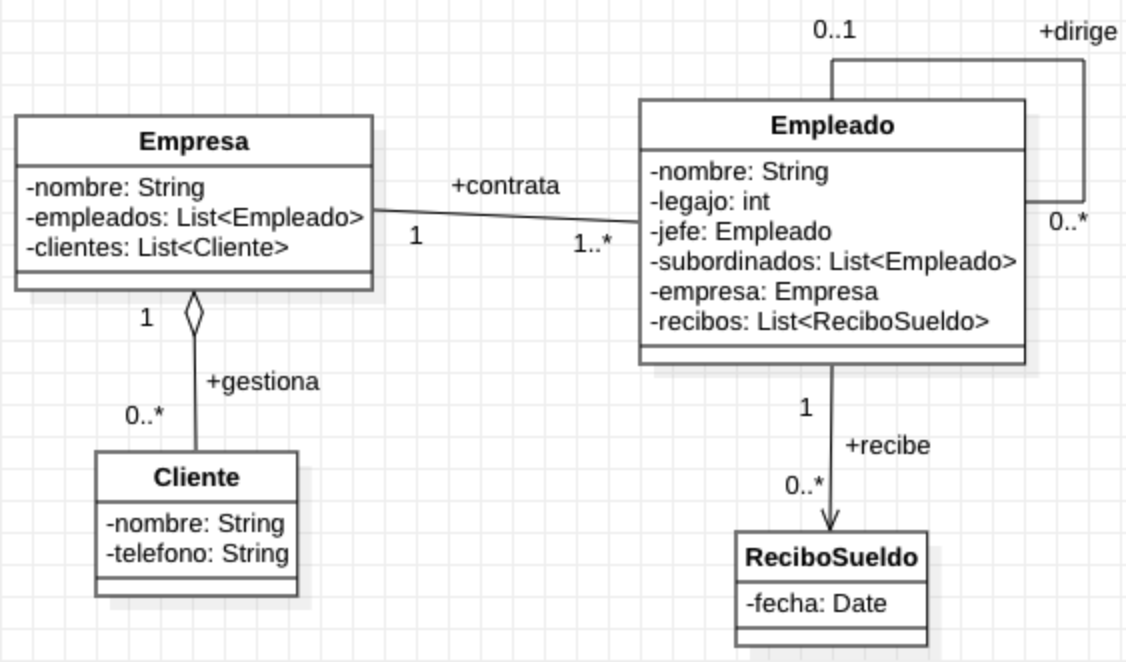
Relación direccional

- Es una relación donde **solo una clase conoce a la otra**
- Hay navegación en **un solo sentido**



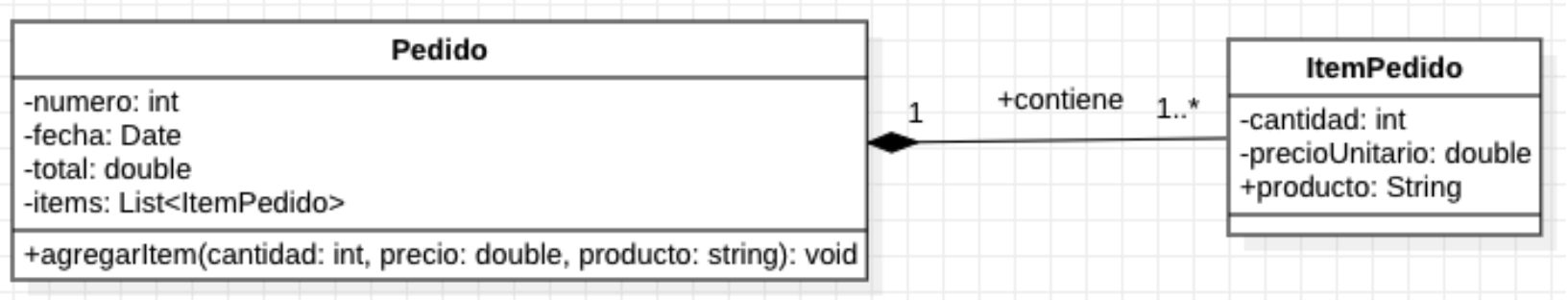
Relación de agregación

- La **agregación** es una relación "**todo–parte**" **débil**.
  - Una clase "**contiene**" a otra
  - Las partes **pueden existir independientemente**



Relación de composición

- La **composición** es una relación **todo–parte fuerte**.
- La parte **NO** puede existir sin el **todo**
- Si el **todo** se **elimina** → las **partes también**

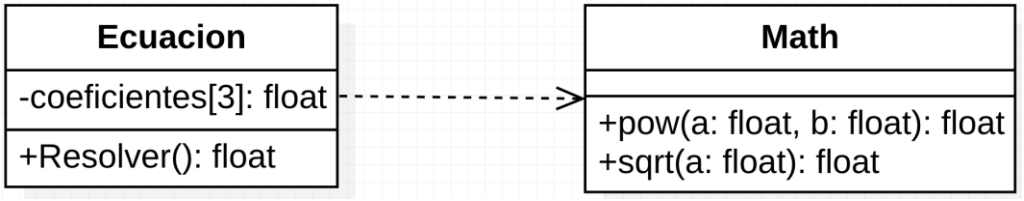


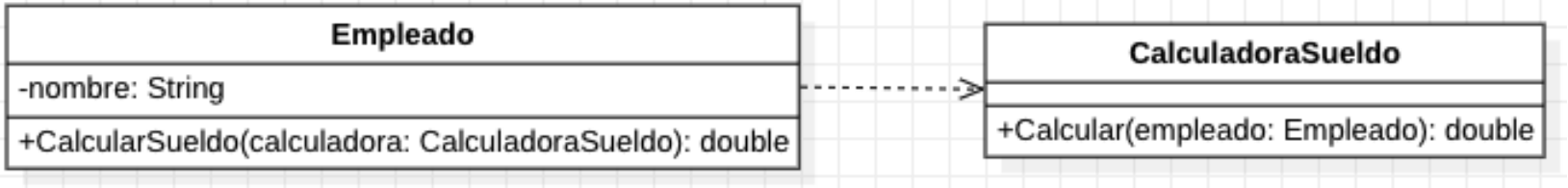
Relación de composición

- **ItemPedido**:
  - ❌ NO existe sin Pedido
  - ❌ NO se comparte entre pedidos
  - ❌ NO tiene identidad propia fuera del pedido
- El lenguaje (Java) **NO obliga** a cumplir composición, sino que se implementa con decisiones de diseño

Relación de dependencia

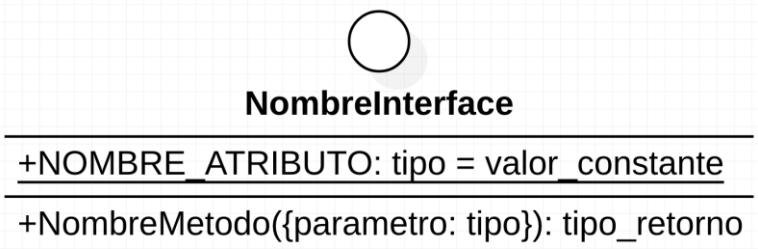
- Una **dependencia** significa:
  - una clase **usa a otra de forma temporal** o **parcial.**





Interfaces

- Una interfaz es una entidad que declara una **serie de atributos, funciones y obligaciones**. Es una especie de contrato donde toda instancia asociada a una interfaz debe de implementar los servicios que indica aquella interfaz.

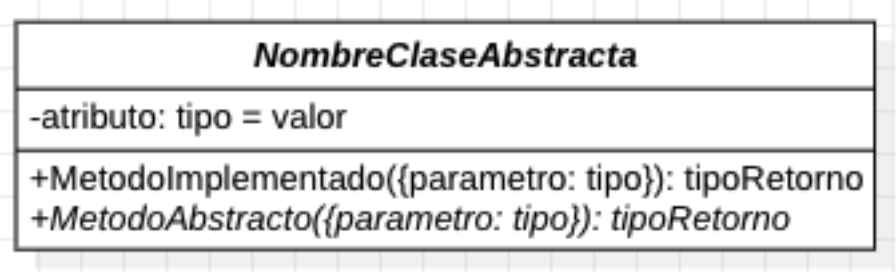


Clases Abstractas

- Una **clase abstracta** es una clase que:
  - **NO se puede instanciar**
  - Sirve como base para otras clases

¿Para qué sirve?

- Evitar que se instancie la clase
- Compartir comportamiento común



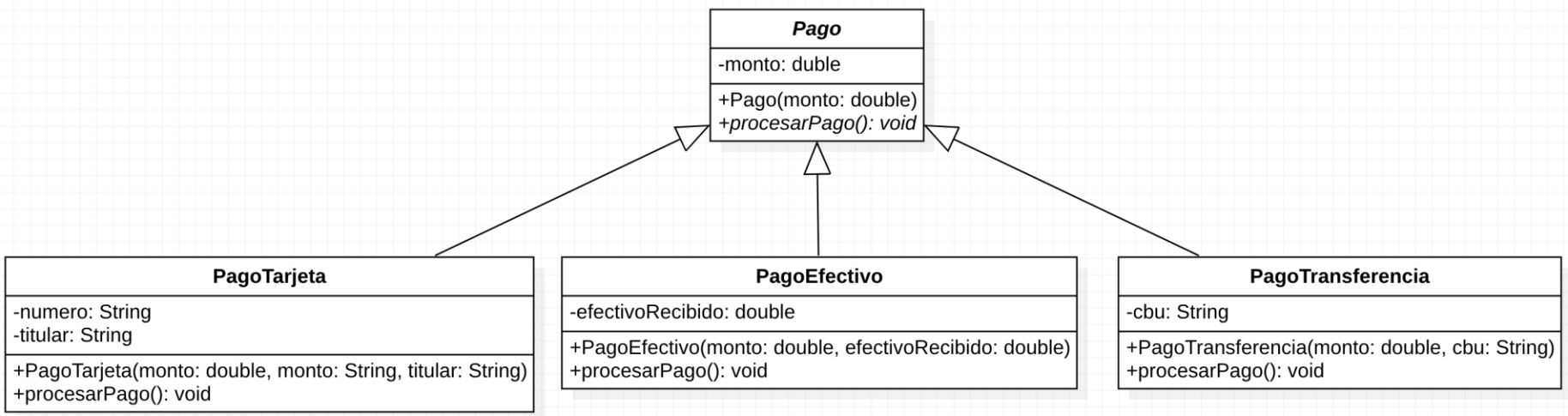
Clases Abstractas vs Interfaz

#	Clase Abstracta	Interfaz
1	Puede heredar o extender cualquier clase (independientemente de que esta sea abstracta o no).	Solamente puede extender o implementar otras interfaces.
2	Puede tener métodos que sean abstractos o que no lo sean.	Sólo pueden definir métodos abstractos.
3	Los métodos abstractos pueden ser públicos o protegidos.	Sólo puede tener métodos públicos.
4	Los atributos pueden ser static, final o static final con cualquier modificador de acceso.	Sólo puede tener constantes public static final.

Generalización (Herencia)

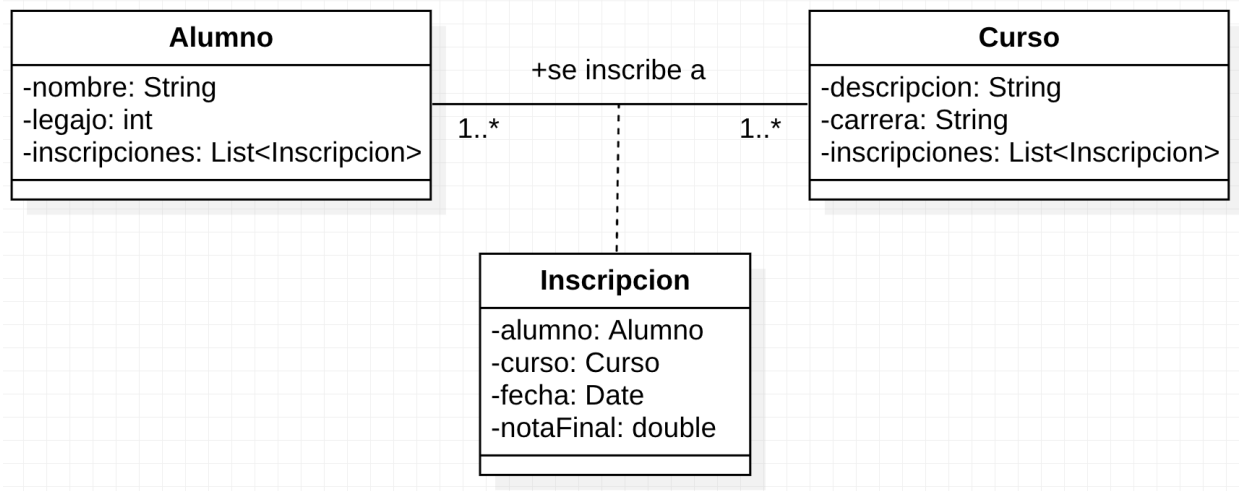
- Son asociaciones entre elementos más generales y elementos más específicos, en los cuales éstos últimos son consistentes totalmente con los primeros, por lo que heredan las características proporcionadas por lo elementos generales y además pueden aumentar información.
- Este tipo de relación también se conoce como **herencia**.
- En una generalización **no** hay **multiplicidad**.
- La visibilidad "***protected***" permite que solo objetos de la misma clase ó subclase vean el elemento.

Herencia


Pago
-monto: duble
+Pago(monto: double)
+procesarPago(): void
PagoTarjeta
-numero: String
-titular: String
+PagoTarjeta(monto: double, monto: String, titular: String)
+procesarPago(): void
PagoEfectivo
-efectivoRecibido: double
+PagoEfectivo(monto: double, efectivoRecibido: double)
+procesarPago(): void
PagoTransferencia
-cbu: String
+PagoTransferencia(monto: double, cbu: String)
+procesarPago(): void

Relación clase de asociación

- Una **clase de asociación** es:
  - una relación entre dos clases que además tiene **atributos propios.**



Clases estáticas

- En realidad, en Java **no existen clases completamente estáticas (como en C#)**, pero sí existen:
  - **Clases con miembros estáticos**
  - **Clases internas estáticas (static nested class)**
- Pero cuando hablamos en la práctica, solemos decir:
- *"una clase estática" = una clase que solo tiene métodos y/o atributos static*

Clases estáticas

- En realidad, en Java **no existen clases completamente estáticas (como en C#)**, pero sí existen:
  - **Clases con miembros estáticos**
  - **Clases internas estáticas (static nested class)**
- Pero cuando hablamos en la práctica, solemos decir:
- *"una clase estática" = una clase que solo tiene métodos y/o atributos static*

Clases estáticas

- Cuando algo es static:
- **Pertenece a la clase, no al objeto**
- ❌ No se necesita hacer ***new***

```java
public class Calculadora {

    public static int sumar(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        int resultado = Calculadora.sumar(5, 3);
        System.out.println(resultado);
    }
}
```

Clases estática anidada

- Es una **clase definida dentro de otra clase**, pero marcada como static.
  - No depende de una instancia de la clase externa
  - Se comporta casi como una clase normal, pero "agrupada" dentro de otra

```
public class ClaseExterna {

    static class ClaseInterna {
        // código
    }
}
```

Ejercicio

La reconocida empresa de servicios de transporte (**Ubir**), nos encarga el diseño de una aplicación que les permita a los usuarios poder realizar sus viajes sin inconvenientes, permitiendo buscar y contratar choferes.

Las funcionalidades que se espera de la aplicación serán:

- Registro y login de usuarios: cada usuario podrá crear una cuenta, la cual podrá ser del tipo **Pasajero** o **Chofer**. Para el caso de los choferes, es indispensable contar con licencia de conducir para lograr un registro exitoso.
- Se dispondrá de una billetera virtual para poder realizar los pagos de los viajes y en donde será depositado (en caso de los choferes), el total del viaje menos la comisión que cobra la plataforma por su uso.
- Dicha billetera podrá ser recargada a través de tarjeta de crédito o PayPal.
- En caso de registrarse como un chofer, será necesario que se ingrese los datos del vehículo que maneja. Es indispensable contar con patente, fecha de VTV, marca y modelo del vehículo.

Asimismo, cada pasajero podrá solicitar un viaje indicando su destino y desde donde comenzará el viaje. Esto avisará a un chofer que esté dentro de un determinado rango (expresado en km). El chofer podrá aceptar/declinar el viaje. En caso de declinar el viaje, el pasajero deberá solicitar uno nuevo.

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

